

Agentic AI

How AI Agents Think, Act, and Remember
using LLMs, Tools, Memory, and RAG

TEACHING MATERIAL · 2025

An AI that decides what to do, not just what to say

Traditional LLM

User asks a question. The model generates one response from its training data. No tools, no memory, no action. Each turn is stateless and isolated.

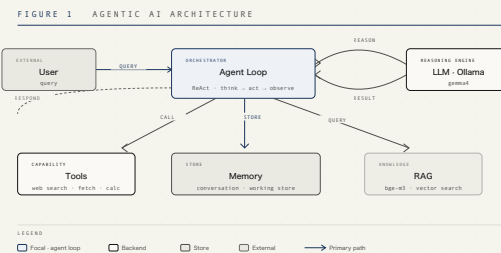
```
User: "What is the weather?"  
LLM: "I can't access real-time  
data. Please check a  
weather website."
```

Agentic AI

The model reasons about what it needs, calls tools to gather data, remembers context across turns, and iterates until it can give a complete answer.

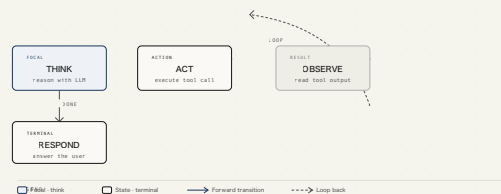
```
User: "What is the weather?"  
Agent: [CALLS weather_api("NYC")]  
→ "75 F, sunny"  
Agent: "It's 75 F and sunny  
in New York right now."
```

Four components working under one agent loop



Every agent decision follows the ReAct pattern

FIGURE 2 THE REACT AGENT LOOP



Four building blocks of an agentic system

A LLM · Reasoning Engine

The brain. Receives context (system prompt, tools, memory, history) and decides: respond directly, or call a tool to gather more information. We use **Ollama + gemma4** running locally.

C Memory · Context Persistence

The notebook. **Conversation history** tracks every message. **Working memory** stores key-value facts the agent saves mid-research. Without memory, every turn is the first conversation.

B Tools · Function Calling

The hands. Functions the LLM can invoke: web search, fetch webpages, save notes, calculate. Each tool is defined with a name, description, and JSON schema so the LLM knows when and how to use it.

D RAG · Knowledge Retrieval

The library. Embeds documents using **bge-m3**, stores vectors in-memory, and retrieves the most relevant chunks by cosine similarity. Turns raw text into searchable knowledge.

Function calling lets the LLM reach beyond its training data

Tool Definition

Each tool has a JSON schema. The LLM reads these and decides which tool to call, with what arguments.

```
{
  "name": "web_search",
  "description": "Search the web
    for information",
  "parameters": {
    "query": {
      "type": "string",
      "description": "Search terms"
    }
  }
}
```

Tool Execution Flow

- 1 LLM outputs a tool call: `web_search("AI agents")`
- 2 Agent loop intercepts and runs the Python function
- 3 Result is fed back to the LLM as a new message
- 4 LLM reasons again: call another tool, or respond

Five tools in our demo: `web_search`, `read_webpage`, `save_to_kb`, `query_kb`, `think`.

Without memory, every turn is the first conversation

Conversation History

Every message — user input, LLM response, tool calls, tool results — is appended to an array. This is the agent's **short-term memory**. The full history is sent to the LLM on every turn, so it never loses context.

```
messages = [
  {"role": "user", "content": "..."},
  {"role": "assistant",
   "tool_call": "web_search(...)"},
  {"role": "tool",
   "result": "..."},
  {"role": "assistant",
   "content": "Based on..." }
]
```

Working Memory

A key-value store the agent writes to during research. Think of it as the agent's **scratchpad** — it saves facts it wants to remember across tool calls.

```
memory = {
  "ai_agents_def": "AI agents
are autonomous systems...",
  "rpa_vs_agentic":
    "RPA follows fixed rules;
agents reason and adapt",
  "search_results": [
    "url1", "url2", "url3"
  ]
}
```

Retrieve relevant knowledge before generating an answer

How It Works

Documents are embedded into vectors using **bge-m3**. When the agent queries the knowledge base, we embed the query and find the most similar document chunks by cosine similarity.

```
1. SAVE: text → embed(text)
   → store(vector, text)
2. QUERY: embed(query)
   → cosine_similarity(query,
      all_vectors)
   → return top-K texts
```

Why RAG Matters for Agents

During research, the agent accumulates findings. RAG turns these into a **searchable knowledge base** — so later queries benefit from earlier discoveries without re-searching the web.

bge-m3	Embedding model (Ollama)	1024-dim vectors
Vector Store	In-memory numpy array	Simple, no DB needed
Similarity	Cosine similarity	Top-K retrieval

An agentic research demo that uses all four capabilities

Here is what happens step-by-step when a user asks: *"Research AI agents and compare them with RPA"*

Step 1	<code>web_search("AI agents vs RPA")</code>	TOOLS
Step 2	<code>read_webpage(result.url)</code>	TOOLS
Step 3	<code>save_to_kb(extracted text) → embed with bge-m3</code>	RAG
Step 4	<code>think("Key finding: agents reason, RPA follows scripts")</code>	MEMORY
Step 5	<code>query_kb("RPA limitations") → retrieve saved facts</code>	RAG
Step 6	LLM synthesizes: search results + KB + memory → final answer	LLM

The agent loop orchestrates every step. Think → Act → Observe. Repeat until the answer is complete.

Agentic AI connects reasoning with action

Traditional LLMs are passive: one prompt, one answer. Agents are active: they reason, decide, call tools, remember, and iterate. Four capabilities make this possible.

LLM	Gemma4 via Ollama — the reasoning engine that decides what to do
Tools	Function calling — search, fetch, save, query, calculate
Memory	Conversation history + working store — context across turns
RAG	bge-m3 embeddings + vector search — knowledge retrieval
Agent Loop	ReAct pattern — Think → Act → Observe → Repeat

LLM thinks. Tools reach. Memory remembers. RAG knows. The loop connects them all.